

KnowledgeAI ? Complete Project Documentation

Version: 1.0
Type: Enterprise AI Knowledge Assistant (SaaS)
Stack: React · FastAPI · MongoDB · Emergent Universal LLM (OpenAI / Claude / Gemini)
Live URL: <https://knowledge-retrieval-3.preview.emergentagent.com>

1. Overview

KnowledgeAI is a production-style SaaS platform that lets employees upload company documents (or connect data sources like GitHub and Notion) and then chat with an AI assistant that retrieves the most relevant content before answering ? a technique called Retrieval-Augmented Generation (RAG). Every answer is grounded in the user's own documents and shows inline citations with the source files and similarity scores.

The product includes a dashboard, a document knowledge base, a ChatGPT-style chat with streaming responses, semantic search, an admin console, configurable model settings, and external connectors ? wrapped in a polished, dark-first UI.

What problem it solves

Employees waste time hunting through scattered documents. KnowledgeAI turns that corpus into an instantly queryable assistant that answers in natural language and always cites where the answer came from, so responses are trustworthy and verifiable.

2. Feature Summary

- | # | Feature | Description |
|----|----------------------|--|
| 1 | Authentication | Google login (Emergent-managed OAuth), secure httpOnly session cookies, Role-Based Access Control (admin / user), profile page. |
| 2 | Dashboard | Total documents, conversations, AI responses, storage used, tokens used, 7-day activity chart, recent chats. |
| 3 | Knowledge Base | Upload PDF, DOCX, TXT, Markdown, CSV, PPTX. Auto text extraction ? chunking ? embeddings ? vector store + metadata. Processing status and delete. |
| 4 | Chat | Streaming responses (SSE), Markdown + code highlighting, copy, regenerate, multiple conversations, history sidebar, model switcher. |
| 5 | RAG | Searches the vector store, retrieves top chunks, injects context into the prompt, generates the answer, and displays citations + source documents. |
| 6 | AI Agent | A lightweight LangGraph-style router that decides whether to search documents or answer directly. |
| 7 | Semantic Search | Search meaning across all documents; shows document name, relevant paragraph, and similarity score. |
| 8 | Admin Dashboard | Manage users, documents, storage, AI usage, average latency, per-model usage chart (admin only). |
| 9 | Settings | Choose provider (GPT-4.1 / Claude / Gemini) and tune temperature, top-p, max tokens. |
| 10 | Connectors | Sync from GitHub (repo files) and Notion (workspace pages), public or private (token). |
| 11 | Monitoring & Logging | Usage logs capture model, provider, token counts, latency, and prompt for every chat. |
| 12 | Security | File-type/size validation, prompt-injection guard, RBAC, server-verified sessions, encrypted transport. |

3. Architecture



```

+-----+-----+-----+-----+-----+-----+
|                                     FastAPI (async)                                     |
| server.py ? routers                                                         |
| +- auth.py           Emergent OAuth, sessions, RBAC deps                     |
| +- routes_docs.py    upload ? extract ? chunk ? embed ? store               |
| +- routes_chat.py    SSE streaming RAG chat, conversations                   |
| +- routes_core.py    dashboard - search - settings - admin                  |
| +- routes_connectors GitHub / Notion sync                                   |
| ai.py               LLM (emergentintegrations) + embeddings + agent router   |
| extract.py          text extraction (pdf/docx/pptx/csv/txt/md) + chunking    |
+-----+-----+-----+-----+-----+
|                                     motor (async)                                     |
+-----+-----+-----+-----+-----+
| MongoDB | users - user_sessions - documents -                               |
|          | chunks(+embedding) - conversations -                             |
|          | messages - usage_logs - user_settings                           |
+-----+-----+-----+-----+-----+

```

Design patterns

- Modular routers / service layer on the backend (clean separation of auth, documents, chat, core, connectors).
- Repository-style data access via dedicated collection handles in db.py.
- Dependency injection through FastAPI Depends (e.g. `get_current_user`, `require_admin`).
- Reusable React hooks (useChat) and presentational components (ConversationSidebar, MessageList, ChatComposer).

4. Technology Stack

Frontend: React, React Router, TanStack React Query, TailwindCSS, shadcn/ui, Recharts, react-markdown + rehype-highlight, lucide-react icons, sonner (toasts).

Backend: Python, FastAPI (async), Motor (async MongoDB driver), Pydantic, httpx, pypdf, python-docx, python-pptx.

AI: Emergent Universal LLM key via emergentintegrations ? supports OpenAI GPT-4.1, Anthropic Claude, and Google Gemini with runtime switching. Embeddings are generated locally with a deterministic feature-hashing vectorizer (offline, no external embedding API required).

Database: MongoDB. Vectors are stored on each chunk document; retrieval uses cosine similarity computed in the service layer.

Deployment: Dockerfiles for frontend and backend + docker-compose.yml (frontend + backend + MongoDB) for one-command local runs.

5. Data Model (MongoDB collections)

Collection | Key fields

users | user_id, email, name, picture, role (admin/user), created_at

user_sessions | user_id, session_token, expires_at, created_at

documents | id, user_id, filename, file_type, size, status, chunk_count, source, created_at

chunks | id, document_id, user_id, filename, chunk_index, text, embedding[]

conversations | id, user_id, title, created_at, updated_at

messages | id, conversation_id, role, content, citations[], model, provider, latency_ms, token_count, created_at

usage_logs | id, user_id, type, model, provider, token_count, latency_ms, prompt, created_at

user_settings | user_id, provider, model, temperature, top_p, max_tokens

6. API Reference (all routes prefixed with /api)

Auth

- POST /auth/session ? exchange Emergent X-Session-ID header for a session cookie
- GET /auth/me ? current authenticated user
- POST /auth/logout ? end session

Documents

- GET /documents ? list the user's documents
- POST /documents/upload ? upload + process a file (multipart, field file)
- DELETE /documents/{id} ? delete a document and its chunks

Chat

- GET /chat/conversations ? list conversations
- POST /chat/conversations ? create a conversation
- GET /chat/conversations/{id}/messages ? messages in a conversation
- DELETE /chat/conversations/{id} ? delete a conversation
- POST /chat/stream ? Server-Sent Events RAG chat (events: meta ? delta ? done)

Search / Settings / Models

- POST /search ? semantic search across documents
- GET/PUT /settings ? read / update model settings
- GET /models ? available models per provider

Dashboard / Admin (admin routes require role = admin)

- GET /dashboard
- GET /admin/overview · GET /admin/users · GET /admin/documents · GET /admin/logs

Interactive OpenAPI docs are available at /docs.

7. RAG & AI Agent Flow

1. The user sends a question on POST /chat/stream.
2. The agent router (route_agent) decides: if the user has documents and the query is substantive, it chooses search; for small talk or when no documents exist, it chooses direct answer.
3. On search: the question is embedded, all of the user's chunks are scored by cosine similarity, and the top 5 chunks are selected as context.
4. A system prompt is built that instructs the model to answer **only** from the provided context and to cite sources as [1], [2], etc.
5. The selected provider/model streams the answer token-by-token over SSE.
6. Citations (filename + similarity score + snippet) are sent first as a meta event and displayed as chips under the answer.
7. The assistant message plus a usage log (model, tokens, latency) are persisted.

Prompt-injection protection: user input is scanned for common override patterns (e.g. "ignore previous instructions") and annotated so the model treats it strictly as a query, not as instructions.

8. Connectors (Data Sources)

GitHub ? provide owner, repo, and branch (optional token for private repos). The connector walks the repo tree, downloads supported files (.md/.txt/.pdf/.docx/.csv/.pptx, capped at 30 files / 15 MB each), extracts text, and indexes them. Re-syncing is idempotent (existing chunks are replaced, no duplicates).

Notion ? provide an integration token; pages shared with the integration are

searched, their block text (including nested blocks) is extracted and indexed.

9. Security

- Authentication: Emergent-managed Google OAuth; session tokens stored as httpOnly, secure, SameSite=None cookies with a 7-day expiry, verified server-side on every request.
- RBAC: admin-only endpoints guarded by a require_admin dependency; the first registered user (or emails in ADMIN_EMAILS) becomes admin.
- File validation: extension allow-list and a 15 MB size limit on uploads.
- Prompt-injection guard: suspicious instruction patterns are neutralized.
- Secrets: all keys live in .env (git-ignored); only .env.example templates are committed.

10. Project Structure

```
/app
+-- backend
|   +-- server.py          # FastAPI app + router registration
|   +-- db.py              # Mongo client + collection handles
|   +-- auth.py            # OAuth session, get_current_user, require_admin
|   +-- ai.py              # LLM chat, embeddings, agent router, prompts
|   +-- extract.py         # file text extraction + chunking
|   +-- routes_docs.py     # document upload / list / delete
|   +-- routes_chat.py     # conversations + SSE streaming chat
|   +-- routes_core.py     # dashboard, search, settings, admin
|   +-- routes_connectors.py # GitHub + Notion sync
|   +-- requirements.txt
|   +-- .env / .env.example
|   +-- tests/             # 33 pytest cases
+-- frontend
|   +-- src
|   |   +-- App.js
|   |   +-- context/AuthContext.jsx
|   |   +-- hooks/useChat.js
|   |   +-- lib/{api,streamChat,format,logger}.js
|   |   +-- components/AppLayout.jsx, ConnectorsBar.jsx
|   |   +-- components/chat/{ConversationSidebar,MessageList,ChatComposer}.jsx
|   |   +-- pages/{Login,AuthCallback,Dashboard,KnowledgeBase,
|   |               ChatPage,SearchPage,SettingsPage,AdminPage,ProfilePage}.jsx
|   +-- public/samples/    # downloadable sample files
|   +-- package.json
|   +-- .env / .env.example
+-- scripts/              # seed + sample-generation scripts
+-- docker-compose.yml
+-- README.md
```

11. Setup & Run

One command (Docker)

```
git clone <your-repo-url> && cd knowledgeai
cp backend/.env.example backend/.env      # set EMERGENT_LLM_KEY
cp frontend/.env.example frontend/.env
docker compose up --build
```

Open <http://localhost:3000> and sign in with Google. The first user becomes admin.

Manual dev

```
# backend
cd backend && pip install -r requirements.txt
uvicorn server:app --host 0.0.0.0 --port 8001 --reload
# frontend
cd frontend && yarn install && yarn start
```

Environment variables

- Backend: MONGO_URL, DB_NAME, CORS_ORIGINS, EMERGENT_LLM_KEY, ADMIN_EMAILS (optional)
- Frontend: REACT_APP_BACKEND_URL

Deployment

Both services are containerized. Push the images to a registry (ECR/ACR/GAR) and deploy to ECS/Fargate, Azure App Service, or Google Cloud Run. Point MONGO_URL at a managed MongoDB (Atlas / DocumentDB / Cosmos), set EMERGENT_LLM_KEY, and set REACT_APP_BACKEND_URL to the backend's public URL.

12. Testing

- 33 backend pytest cases cover auth + RBAC, upload/list/delete, RAG streaming across all three providers, connector re-sync idempotency, semantic search, settings persistence, and dashboard aggregates.
- Frontend E2E (Playwright) validated the full flows: dashboard, chat streaming with citations, copy/regenerate, model switching, multiple conversations, semantic search, settings, admin, and connector sync ? with zero console errors.

```
cd backend && pytest tests/ -v
```

13. How to Use (end-user quick start)

1. Sign in with Google.
2. Add knowledge ? go to *Knowledge Base* and upload files, or click *Connect a Source* to sync a GitHub repo or Notion workspace.
3. Ask ? open *Chat*, type a question, and read the streamed answer with [1] citations. Switch models (GPT-4.1 / Claude / Gemini) from the top-right.
4. Search ? use *Semantic Search* to find the exact paragraphs across all docs.
5. Tune ? in *Settings*, pick your model and adjust temperature / top-p / max tokens.
6. Admin ? admins can review users, documents, and AI usage metrics.

14. Notes & Future Enhancements

- Embeddings are computed locally (deterministic feature-hashing) so retrieval works fully offline; swapping in a hosted embedding API would improve semantic fidelity.
- Roadmap ideas: additional connectors (Confluence, Slack, Google Drive, SharePoint), scheduled auto-sync, per-request LLM cost tracking, light theme, voice input/output, conversation export & sharing, bookmarks, and search history.

Document generated for the KnowledgeAI project. Built on the Emergent platform.